

```

#include <iostream>
#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <sstream>
#include <map>
#include <termios.h>
#include <chrono>
#include <pigpio.h>

#include "sl_lidar.h"
#include "sl_lidar_driver.h"

using namespace sl;
ILidarDriver* lidar = nullptr;

// -----
// millis() Linux
// -----
unsigned long millis() {
    using namespace std::chrono;
    static steady_clock::time_point start = steady_clock::now();
    return (unsigned long)duration_cast<std::chrono::milliseconds>(
        std::chrono::steady_clock::now() - start
    ).count();
}

// -----
// Serial config
// -----
void configSerialPort(int fd) {
    struct termios tty;
    tcgetattr(fd, &tty);
    cfmakeraw(&tty);
    cfsetispeed(&tty, B115200);
    cfsetospeed(&tty, B115200);
    tty.c_cflag &= ~CRTSCTS;
    tty.c_cflag |= CREAD | CLOCAL;
    tty.c_cflag &= ~PARENB;
    tty.c_cflag &= ~CSTOPB;
    tty.c_cflag &= ~CSIZE;
    tty.c_cflag |= CS8;
    tty.c_cc[VTIME] = 0;
    tty.c_cc[VMIN] = 0;
    tcflush(fd, TCIFLUSH);
    tcsetattr(fd, TCSANOW, &tty);
}

```

```

// -----
// Ctrl+C handler
// -----
void ctrlc_handler(int) {
    if (lidar) {
        lidar->stop();
        lidar->reset();
        delete lidar;
        lidar = nullptr;
    }
    gpioTerminate();
    exit(0);
}

// =====
// GROVE ULTRASONIC RANGER READING
// =====
float readGroveUltrasonic(int pin)
{
    // Trigger pulse
    gpioSetMode(pin, PI_OUTPUT);
    gpioWrite(pin, 0);
    gpioDelay(2);

    gpioWrite(pin, 1);
    gpioDelay(10); // 10 µs
    gpioWrite(pin, 0);

    // Switch to input to read pulse width
    gpioSetMode(pin, PI_INPUT);

    // Wait for pulse to go HIGH
    uint32_t start_wait = gpioTick();
    while (gpioRead(pin) == 0) {
        if (gpioTick() - start_wait > 30000) return -1;
    }

    // Measure HIGH pulse width
    uint32_t high_start = gpioTick();
    while (gpioRead(pin) == 1) {
        if (gpioTick() - high_start > 30000) return -1;
    }
    uint32_t high_end = gpioTick();

    uint32_t duration = high_end - high_start;

    // Grove formula: distance (cm) = duration / 58
    return duration / 58.0f;
}

```

```

}

// =====
// MAIN
// =====
int main() {
    signal(SIGINT, ctrlc_handler);

    // ---- pigpio init ----
    if (gpioInitialise() < 0) {
        std::cerr << "pigpio init failed!" << std::endl;
        return 1;
    }

    // ---- LIDAR init ----
    auto channel_res = createSerialPortChannel("/dev/ttyUSB0", 256000);
    IChannel* channel = *channel_res;

    auto driver_res = createLidarDriver();
    lidar = *driver_res;

    lidar->connect(channel);
    lidar->reset(5000);
    usleep(3000000);

    lidar->setMotorSpeed();
    lidar->startScan(false, false);
    lidar->setMotorSpeed(200);

    sl_lidar_response_measurement_node_hq_t nodes[8192];
    size_t count = sizeof(nodes)/sizeof(nodes[0]);

    const float FLOOR_OBSTACLE_THRESHOLD_CM = 30.0f; // tune this

    while (true) {
        float sum_dist[360] = {0};
        int count_dist[360] = {0};

        count = sizeof(nodes)/sizeof(nodes[0]);
        lidar->grabScanDataHq(nodes, count, 0);
        lidar->ascendScanData(nodes, count);

        float avg0 = 0;
        float maxValue = 0;
        int maxAngle = 0;

        for (size_t i = 0; i < count; ++i) {
            int angle = (nodes[i].angle_z_q14 * 90) >> 14;

```

```

float dist = nodes[i].dist_mm_q2 / 4.0f;

if ((angle >= 270 && angle < 360) || (angle >= 0 && angle <= 90)) {
    sum_dist[angle] += dist;
    count_dist[angle]++;
}
}

for (int a = 0; a < 360; ++a) {
    if (count_dist[a] > 0) {
        float avg = sum_dist[a] / count_dist[a];
        if (a == 0) avg0 = avg;
        if (avg > maxAngle) {
            maxAngle = avg;
        }
    }
}

int sendAngle = (avg0 > 500) ? 0 : maxAngle;
int speed = 1500;

if (avg0 > 500) speed = 1680;
else if (avg0 >= 300) speed = 1650;
else if (avg0 >= 200) speed = 1630;
else if (avg0 >= 100) speed = 1620;
else {
    speed = 1350;
    sendAngle = (sendAngle + 180) % 360;
}

// ---- GROVE ULTRASONIC FLOOR SENSOR ----
float floorDist = readGroveUltrasonic(22);

bool floorObstacle = (floorDist > 0 && floorDist <
FLOOR_OBSTACLE_THRESHOLD_CM);

if (floorObstacle) {
    std::cout << "[ULTRA] obstacle on floor at " << floorDist << " cm";
    if (sendAngle == 0) {
        sendAngle = 45;
    }
    std::cout << " | angle=" << sendAngle << std::endl;
    speed = 1300;
} else {
    std::cout << "[LIDAR] dist_front=" << avg0
        << " | angle=" << sendAngle
        << " | speed=" << speed

```

```

        << (speed < 1500 ? " (REVERSE)" : " (FORWARD)")
        << std::endl;
    }

    // === JSON ===
    std::stringstream ss;
    ss << "{\"action\":\"drive\", \"angle\":" << sendAngle
        << ", \"speed\":" << speed << "}";

    int serial_port = open("/dev/ttyUSB1", O_RDWR | O_NOCTTY | O_NDELAY);
    if (serial_port >= 0) {
        fcntl(serial_port, F_SETFL, 0);
        configSerialPort(serial_port);
        std::string msg = ss.str() + "\n";
        write(serial_port, msg.c_str(), msg.size());
        close(serial_port);
    }

    usleep(50000);
}

gpioTerminate();
return 0;
}

```